

COMPTE RENDU DE TP07 – Bataille Navale

Nom : Duchesne-Brajon Délio

Groupe : 1

COMPTE RENDU DE TP07 – Bataille Navale

1. Analyse du problème

2. Conception de la solution

2.1. Découpage du problème

2.2. Structures de données

2.3. Algorithmes principales fonctions

Initialisation

Placement du joueur

Placement de l'ordinateur

Affichage

Tir du joueur

Tir de l'ordinateur

Fin de partie

3. Réalisation

3.1. Environnement

3.2. Gestion des entrées / sorties

4. Tests

4.1. Méthode de test

4.2. Cas de tests

Test 1 : placement correct des pions du joueur

Test 3 : tir touché

Test 4 : tir raté

Test 5 : victoire du joueur

Test 6 : rejouer

1. Analyse du problème

L'objectif du TP est de programmer en Java un jeu simplifié de bataille navale sur une grille de 5×5 cases.

Le joueur et l'ordinateur placent chacun 5 pions.

Le but est de toucher tous les pions adverses en jouant alternativement.

Les états possibles d'une case sont :

- **0** : case vide
- **1** : pion présent
- **2** : pion touché
- **3** : tir dans l'eau

Le jeu se termine lorsque l'un des deux joueurs a trouvé les 5 pions de l'autre.

2. Conception de la solution

2.1. Découpage du problème

Pour organiser le programme, le TP est divisé en plusieurs étapes :

1. Initialiser les grilles du joueur et de l'ordinateur.
2. Placer les pions du joueur.
3. Placer les pions de l'ordinateur.
4. Afficher la grille du joueur.
5. Afficher la grille cachée de l'ordinateur.
6. Gérer les tirs du joueur.
7. Gérer les tirs de l'ordinateur.
8. Tester la fin de partie et rejouer.

2.2. Structures de données

Nous utilisons deux tableaux 2D d'entiers :

```
int[][] tabjoueur = new int[5][5];  
int[][] tabordi = new int[5][5];
```

Ils représentent l'état des cases selon la valeur (0,1,2,3).

2.3. Algorithmes principales fonctions

Initialisation

Deux boucles imbriquées mettent toutes les cases à 0.

Placement du joueur

Saisir ligne + colonne → vérifier → placer un pion (1) si valide.

Placement de l'ordinateur

Générer ligne + colonne aléatoires → vérifier → placer un pion.

Affichage

- Grille visible du joueur : on affiche tout.
- Grille cachée de l'ordi : on cache les cases non jouées.

Tir du joueur

- Saisie ligne + colonne
- Si pion → passer la case à 2
- Sinon → passer à 3

Tir de l'ordinateur

Même principe mais en aléatoire.

Fin de partie

La partie se finit quand un joueur a trouvé 5 pions adverses.

```

    for (int i = 0; i < 5; i++) {
        for (int y = 0; y < 5; y++) {
            tabjoueur[i][y] = 0;
            tabordi[i][y] = 0;
        }
    }
}

```

```

for (int n = 1; n <= 5; n++) {
    System.out.println("Placement du pion n°" + n);
    positionValide = false;

    while (!positionValide) {
        System.out.print("Choisissez une ligne (0 à 4) : ");
        ligne = scanner.nextInt();

        System.out.print("Choisissez une colonne (0 à 4) : ");
        colonne = scanner.nextInt();

        if (ligne < 0 || ligne >= 5 || colonne < 0 || colonne >= 5) {
            System.out.println("Position hors du tableau. Réessayez.");
        } else if (tabjoueur[ligne][colonne] != 0) {
            System.out.println("Case déjà occupée. Réessayez.");
        } else {
            tabjoueur[ligne][colonne] = 1;
            System.out.println("Pion placé en [" + ligne + "][" + colonne + "]\n");
            positionValide = true;
        }
    }
}

```

3. Réalisation

3.1. Environnement

Le programme est réalisé en Java avec Eclipse.

La classe principale contient :

- l'initialisation,
- la boucle de jeu,
- les appels aux fonctions.

3.2. Gestion des entrées / sorties

- Saisies : `Scanner`
- Aléatoire : `Random`
- Affichages : `System.out.println`

3.3. Correspondance conception → code

Chaque méthode correspond aux sous-parties de la conception.
Le code suit exactement le découpage du TP.

4. Tests

4.1. Méthode de test

Les tests sont faits en exécutant le programme plusieurs fois.
On observe la console pour vérifier les saisies, tirs, victoires, etc.

4.2. Cas de tests

Test 1 : placement correct des pions du joueur

Le joueur entre 5 coordonnées valides.

	1	2	3	4	5
1	~	~	~	~	~
2	~	~	o	o	o
3	~	~	~	o	~
4	~	~	~	~	o
5	~	~	~	~	~

Test 3 : tir touché

Le joueur vise un pion de l'ordinateur.

```
else if (tabordi[ligne][colonne] == 1) {  
    System.out.println("Touché !");  
}
```

Test 4 : tir raté

Le joueur tire dans l'eau.

```
À vous de jouer !  
Choisissez une ligne (1 à 5) : 3  
Choisissez une colonne (1 à 5) : 4  
Tir en cours...  
Raté !
```

Test 5 : victoire du joueur

Le joueur découvre les 5 pions.

```
Vous avez gagné ! Tous les pions ennemis sont touchés !");
```

Test 6 : rejouer

Le joueur répond "o" pour relancer une partie.

```

if (nbPionTrouveJoueur == 5) {
    System.out.println("\n Vous avez gagné ! Tous les pions ennemis sont touchés !");
    break;
}

if (nbPionTrouveOrdi == 5) {
    System.out.println("\n L'ordinateur a gagné ! Tous vos pions ont été détruits !");
    break;
}

System.out.print("\nVoulez-vous rejouer ? (o/n) : ");
char reponse = scanner.next().toLowerCase().charAt(0);
if (reponse == 'o') {
    System.out.println("\nNouvelle partie !");
    main(args); // relance le jeu
} else {
    System.out.println("Fin du jeu. Merci d'avoir joué !");
}

```